

NAME:Geethanjali
2303A51005

Connect to a Weather API to fetch current weather details for a given city.

- Prompt AI to generate Python code using the requests library.
- Handle errors: invalid city name, API key errors, and network failures.
- Display temperature, humidity, and weather description in a readable format.

- A working Python script that fetches and displays weather data with proper error handling.

```
Ass1.py x
Ass1.py > ...
1 '''
2 Modify the weather API Python script to take the city name as user input from the console and display the weather details.
3 '''
4 import requests
5 def get_weather(city):
6     api_key = 'your_api_key_here' # Replace with your actual API key
7     base_url = 'http://api.openweathermap.org/data/2.5/weather'
8     params = {
9         'q': city,
10        'appid': api_key,
11        'units': 'metric'
12    }
13    response = requests.get(base_url, params=params)
14    if response.status_code == 200:
15        data = response.json()
16        weather_description = data['weather'][0]['description']
17        temperature = data['main']['temp']
18        humidity = data['main']['humidity']
19        print(f"Weather in {city}: {weather_description}")
20        print(f"Temperature: {temperature}°C")
21        print(f"Humidity: {humidity}%")
22    else:
23        print(f"Error fetching weather data for {city}. Please check the city name and try again.")
24 if __name__ == "__main__":
25     city_name = input("Enter the city name to get the weather details: ")
26     get_weather(city_name)
```

Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.

- Use AI to generate API request code.
- Validate user input for amount and currency codes.
- Implement error handling for invalid responses and API downtime.

- Show conversion results clearly in tabular format.

Expected Output:

- A script that converts INR to multiple currencies with graceful error handling.

The screenshot shows a VS Code editor with a Python script named `ass2.py` and its terminal output. The script uses the `requests` library to fetch exchange rates from a free API. It includes input validation for the amount and handles various exceptions like `Timeout`, `ConnectionError`, and `Exception`. The terminal output shows the user entering '200' and the script displaying a table of currency conversions for INR.

```
1 """
2 "Generate a Python script to convert INR to multiple currencies using an API, with input validation, error handling, and tabular output."
3 """
4 import requests
5
6 # API URL (Free exchange rate API)
7 API_URL = "https://api.exchangerate-api.com/v4/latest/INR"
8
9 # Supported currencies
10 SUPPORTED_CURRENCIES = ["USD", "EUR", "GBP", "JPY", "AUD"]
11
12 def get_exchange_rates():
13     try:
14         response = requests.get(API_URL, timeout=5)
15
16         if response.status_code != 200:
17             print(" Error: Unable to fetch data from API.")
18             return None
19
20         return response.json()
21
22     except requests.exceptions.Timeout:
23         print(" Error: Request timed out.")
24     except requests.exceptions.ConnectionError:
25         print(" Error: Network Issue.")
26     except Exception as e:
27         print(f" Unexpected error: {e}")
28
29     return None
30
31
32 def validate_amount(amount):
33     try:
34         value = float(amount)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 /Users/geethanjali/Desktop/AI/ass2.py
geethanjali@geethanjali: AI % ./usr/local/bin/python3 /Users/geethanjali/Desktop/AI/ass2.py
Enter amount in INR: 200

1) Currency Conversion (INR)
Currency      Amount
-----
USD           2.12
EUR           1.24
GBP           1.59
JPY          336.86
AUD           3.06
```

% geethanjali@geethanjali AI %

Task 3 – GitHub Repository Info Fetcher

Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

Instructions:

- Prompt AI to write a Python function to send GET requests to GitHub API.
- Handle rate limit errors, 404 (repo not found), and invalid input.
- Display repository name, description, stars, forks, and open issues.

Expected Output:

- A Python program that retrieves and displays GitHub repository information with robust error handling.

```

1 """
2 "Generate a Python program using requests to fetch GitHub repository details (stars, forks, issues) with input validation and error handling for 404, rate limits, and network errors"
3 """
4
5 import requests
6
7 def get_repo_info(owner, repo):
8     url = f"https://api.github.com/repos/{owner}/{repo}"
9
10    try:
11        response = requests.get(url, timeout=5)
12
13        # Handle errors
14        if response.status_code == 404:
15            print("Error: Repository not found.")
16            return
17        elif response.status_code == 403:
18            print("Error: Rate limit exceeded. Try again later.")
19            return
20        elif response.status_code != 200:
21            print(f"Error: Unexpected response ({response.status_code})")
22            return
23
24        data = response.json()
25
26        # Extract details
27        name = data.get("full_name", "N/A")
28        description = data.get("description", "No description")
29        stars = data.get("stargazers_count", 0)
30        forks = data.get("forks_count", 0)
31        issues = data.get("open_issues_count", 0)
32
33        # Display output
34        print("\n GitHub Repository Info")
35        print("-----")
36        print(f"Name       : {name}")
37        print(f"Description : {description}")
38        print(f"★ Stars    : {stars}")
39        print(f"✂ Forks    : {forks}")
40        print(f"🐛 Issues   : {issues}")
41
42    except requests.exceptions.Timeout:
43        print("Error: Request timed out.")
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

% python3 get_repo_info.py
Enter repository (owner/repo): python/python
Enter Repository (owner/repo): python/python

 GitHub Repository Info
-----
Name       : python/python
Description : The Python programming language
★ Stars    : 7108
✂ Forks    : 3289
🐛 Issues   : 324

```

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario: Build a News Aggregator using a free News API.

Requirements:

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.
- Include a retry mechanism if the first request fails.

Expected Output:

- A working Python script that retrieves and displays news headlines with proper error handling

```
Ass1.py  Ass2.py  Ass3.py  Ass4.py x
Ass4.py > ...
1  """
2  Generate a Python script using requests to fetch top 5 news headlines by category with input validation, retry mechanism, and error handling f
3  """
4  import requests
5  def get_news(category):
6      api_key = 'your_api_key_here' # Replace with your actual API key
7      base_url = 'https://newsapi.org/v2/top-headlines'
8      params = {
9          'category': category,
10         'apiKey': api_key,
11         'country': 'us',
12         'pageSize': 5
13     }
14     try:
15         response = requests.get(base_url, params=params, timeout=5)
16         if response.status_code == 200:
17             data = response.json()
18             articles = data.get('articles', [])
19             if not articles:
20                 print(f"No news found for category: {category}")
21                 return
22             print(f"Top 5 news headlines in '{category}' category:")
23             for idx, article in enumerate(articles, start=1):
24                 print(f"{idx}. {article['title']}")
25         elif response.status_code == 401:
26             print("Error: Invalid API key. Please check your API key and try again.")
27         elif response.status_code == 400:
28             print("Error: Invalid category. Please enter a valid category (e.g., business, entertainment, general, health, science, sports, technology).")
29         else:
30             print(f"Error: Unexpected response ({response.status_code})")
31     except requests.exceptions.Timeout:
32         print("Error: Request timed out. Please try again later.")
33     except requests.exceptions.ConnectionError:
34         print("Error: Network problem. Please check your connection and try again.")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python + - [ ] [ ] [ ] [ ] [ ] x
/usr/local/bin/python3 /Users/geethanjali/Desktop/AI/Ass4.py
geethanjali@geethanjali AI % /usr/local/bin/python3 /Users/geethanjali/Desktop/AI/Ass4.py
Enter news category (business, entertainment, general, health, science, sports, technology): general
Error: Invalid API key. Please check your API key and try again.
geethanjali@geethanjali AI %
```